Q: any questions?v

Overlay Networks and Peer-to-Peer

COMP2014 Distributed Systems

Chris Greenhalgh

School of Computer Science

Contents

- Overlay network
- Peer-to-Peer
- P2P Examples
  - Napster
  - Gnutella
  - Skype
  - Pastry (DHT)

Overlay Networks (Section)

Overlay networks

- The Internet = generic universal communication service
  - Not tailored to specific requirements or situations
- **Overlay network** = **virtual network**
  - Built on top of an existing network, e.g. the Internet
  - Approximately extra layers in protocol stack
- Multiple overlay networks can exist on an existing network
  - Each involving only certain hosts and
  - **specialised** for specific purposes…

Types of overlay networks

Q: which have you used?

- Tailored to a **specific application** or service:
  - Distributed hash tables (see later)
  - Peer-to-peer file sharing (see later)
  - Content distribution networks
- Tailored for **challenging network** environments:
  - Wireless ad hoc networks
  - Delay/disruption tolerant networks
- Offering additional **network features**:
  - (Wide area) Multicast, e.g. MBone (see multicast lecture)
  - Resilience
  - Security, e.g. VPNs (see security lecture), TOR

Overlay routing vs IP routing

IP

- Includes every node
- Uses IP addresses
- Fixed address assignment to nodes
- No replication
- Stable network topology
- Fixed (next hop) routing
- Simple local join (DHCP, etc.)
- Must trust whole network
- No anonymity for address owners

Overlay network

- Includes specific nodes/processes
- Defines own addresses, e.g. GUIDs
- Addresses may be assigned to objects/data
- Often replicated
- Often volatile network
- Overlay-defined routing
- Overlay network specific join
- Need not trust whole network
- Can have anonymity for address owners

Overlay networks and peer-to-peer (1/2)

An overlay network is effectively another communication paradigm…

- A peer-to-peer system can be seen as
  - a set of (equivalent) processes
  - communicating via a dedicated **application-specific overlay network**
- The overlay network provides the communication **services** needed by the peer-to-peer application
  - (See later examples)

Peer-to-Peer (Section)

Peer-to-peer systems

- **Client-server** approach:
  - The server(s) are owned and managed by single service provider

  $\Rightarrow$ Scale limited by cost of administration and operation

  $\Rightarrow$ Limited bandwidth to a single set of machines

  (hence CDNs and other distributed hosting)
- Peer-to-peer aims to be completely **decentralised**
  - 'applications that exploit resources available at the **edges** of the Internet – storage, cycles, content, human presence'. [Shirky 2000]

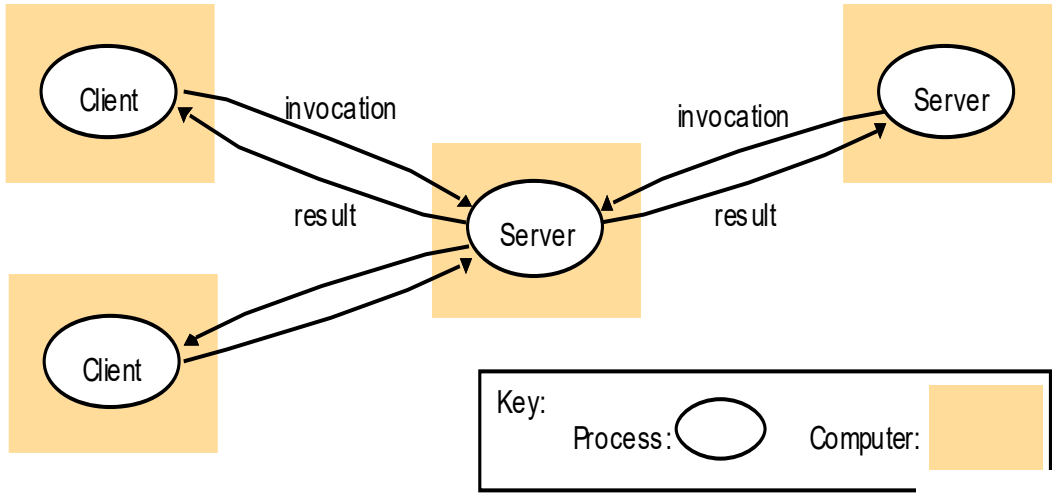


Figure 2.3
Clients invoke individual servers

(client-server vs peer-to-peer)

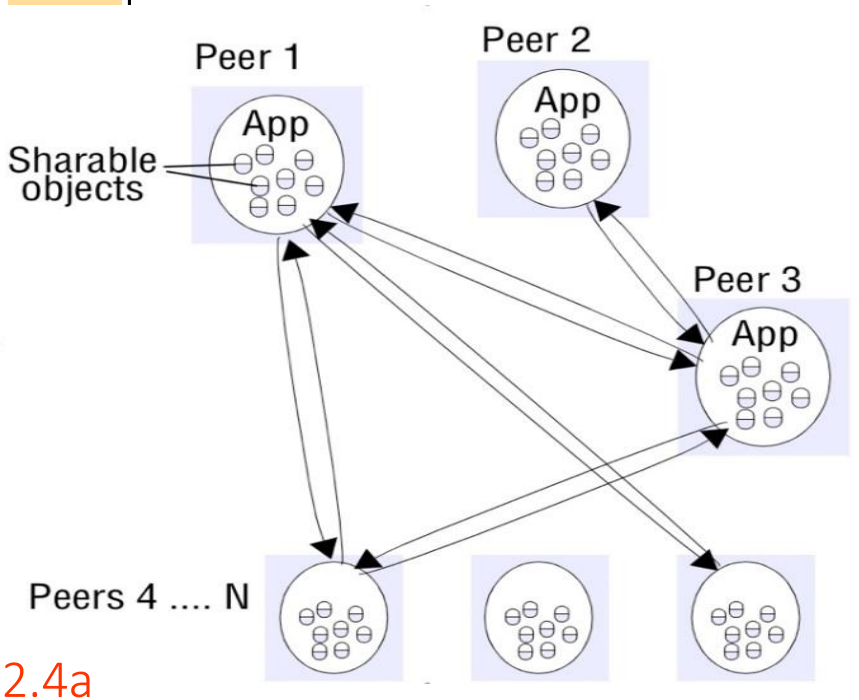


Figure 2.4a
Peer-to-peer architecture

Examples – applications and systems

- **File sharing**
  - Napster (music – see later); Freenet, Gnutella (see later), Kazaa, BitTorrent
- **Teleconferencing**
  - Skype (see later)
- Multiplayer (LAN) **gaming**
  - DOOM (see next lecture), …
- P2P **Middleware** (research toolkits)
  - Pastry (see later), Tapestry, OceanStore
    - Web cacheing, file store

Q: any other P2P examples?

Characteristics of peer-to-peer systems

- Each user (node) **contributes** resources
- All nodes have the **same** functional capabilities and responsibilities
- Correct operation does **not** depend on any **centrally** administered system
- Note:
  - Can offer limited autonomy to providers and users of resources
  - Algorithms for placement of data and subsequent access are critical

Typical P2P Goals
(depending on application/context)

- Low/no centralised resource requirements & **cost**
- Global **scalability**
- **Load balancing**
- Optimization for **local** interactions between neighbouring peers
- Coping with highly **dynamic** host availability
- **Security** of data in an environment with heterogeneous trust
- **Anonymity**, **deniability** and resistance to **censorship**
  - Depending on specific application/scenario

Overlay networks and peer-to-peer (2/2)

- (Since) A peer-to-peer system can (usually) be seen as
  - a set of (equivalent) processes
  - communicating via a dedicated **application-specific overlay network**
- => solve the (simpler) sub-problems:
  - What communication **services** should the overlay network provide?
  - How do processes **join/leave** the overlay network?
    - And how does the overlay network cope with process failures?
  - How can the individual processes **use** these communication services to realise the application?
    - E.g. where should data be stored and how should it be accessed?

Bootstrapping nodes

Q: any ideas?

I.e. how does a node find an overlay network or peer in the first place? It depends…

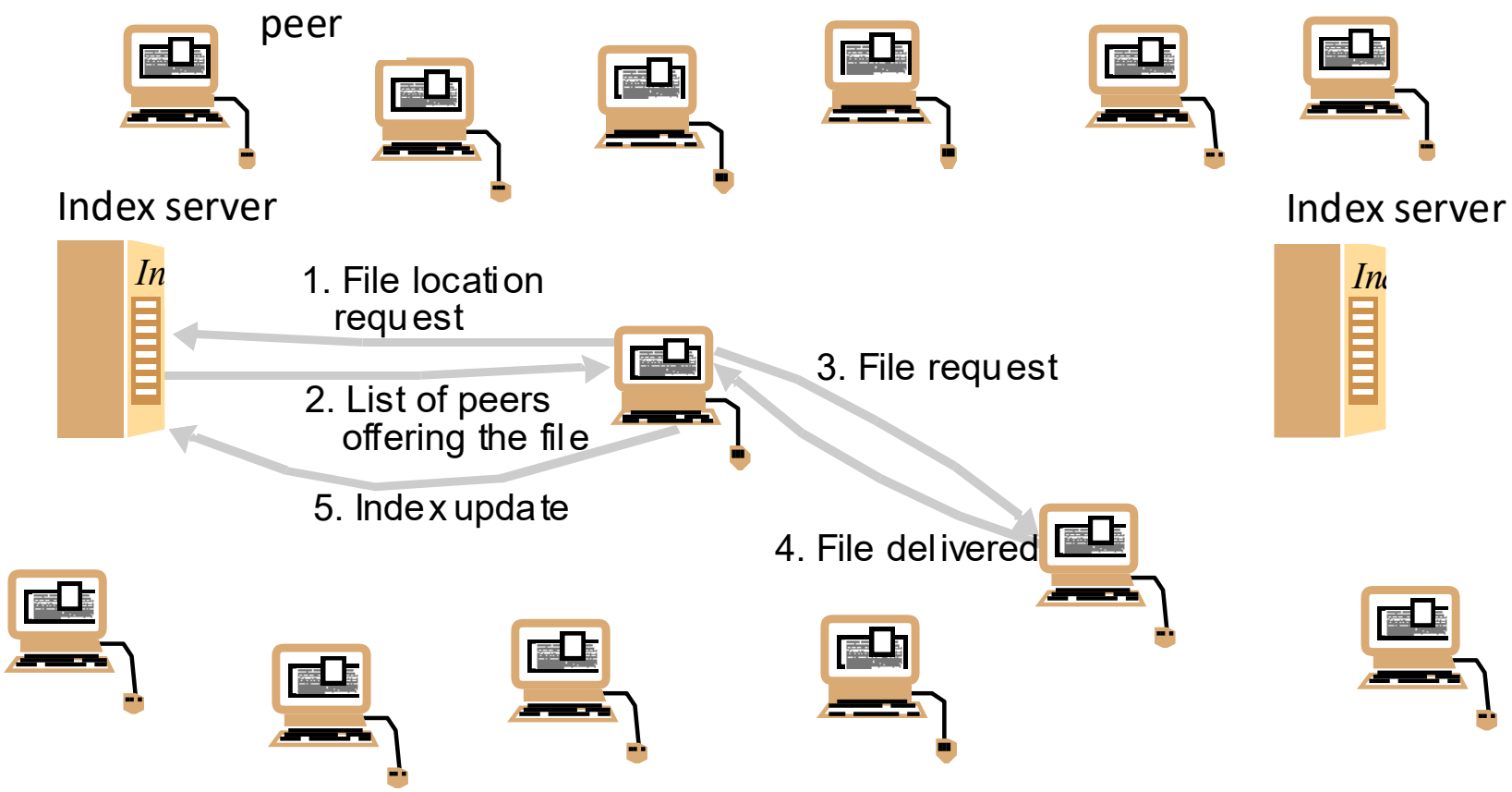
- Network links are inherently local? E.g. wireless ad hoc network, LAN-only application
  - => Local network discovery – see next lecture
- Hybrid of client-server & peer-to-peer over the Internet? E.g. Napster, Skype
  - => Contact server first, which will maintain a list of (some) peers
- Full peer-to-peer over the Internet? E.g. TOR, Bittorrent
  - => (if not first join) try the same IP addresses as last time
  - => Get initial contact information from somewhere else (e.g.) a website
  - => Resolve a hard-code DNS name to get an initial list of rendezvous/peer IP addresses and/or
  - => Use a set of hard-coded IP addresses of “reliable” rendezvous/peers

Peer-to-Peer Examples

Napster

- Peer-to-peer became feasible with the widespread availability of always-on home broadband
  - USA ~1999
- Napster – first generation peer-to-peer system
- Music exchange, i.e. allowed people to share mp3 files
- Not a pure peer-to-peer system
  - Depended on **centralised** (replicated) indexes
  - But actual file storage & transfer only between **peers**

Napster: peer-to-peer file sharing with a centralized, replicated index



Notes

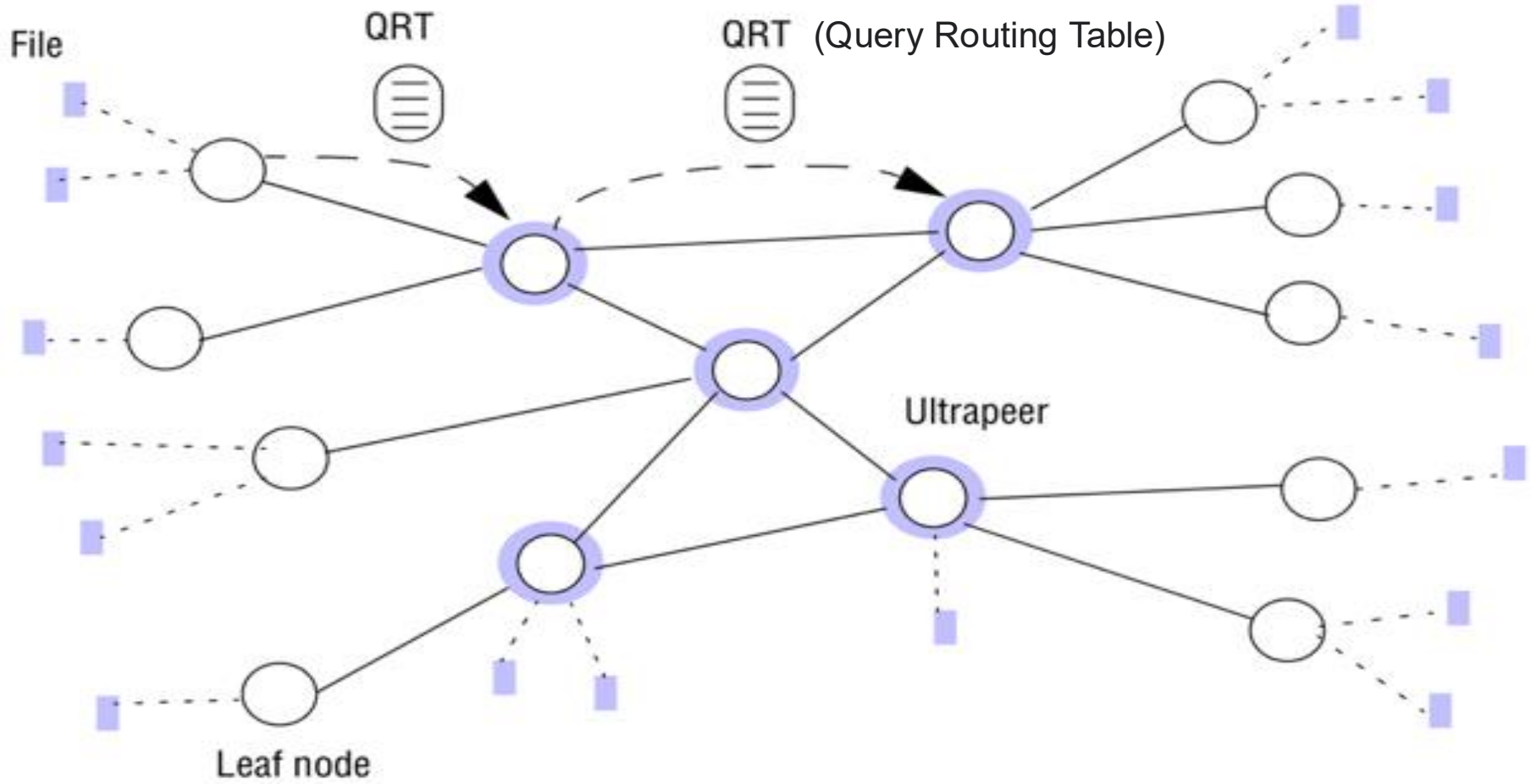
- Music files don't change
  - Can be identified by a **secure hash** of the binary value
  - Looked up in **index**, together with node(s) having copies
- When a node downloads a copy from a peer it is expected to make that copy available to other peers
  - Capacity of network increases as nodes **join**
  - Popular files automatically **replicated**
    - New requests can be served from new replicas
- Illegal:
  - It could not operate without the central servers

Q: why did Napster lose?

Gnutella

- Second generation peer-to-peer file sharing service
  - No central services/indexes
  - But still not “pure” peer-to-peer…
- Two-tier system where some nodes are elected “ultrapeers”
  - **Ultrapeers** form the core network
    - Highly inter-connected
    - Have global IP addresses, i.e. not behind NAT
  - Other nodes act as **leaf** nodes, connecting to ultrapeers

Key elements in the Gnutella protocol



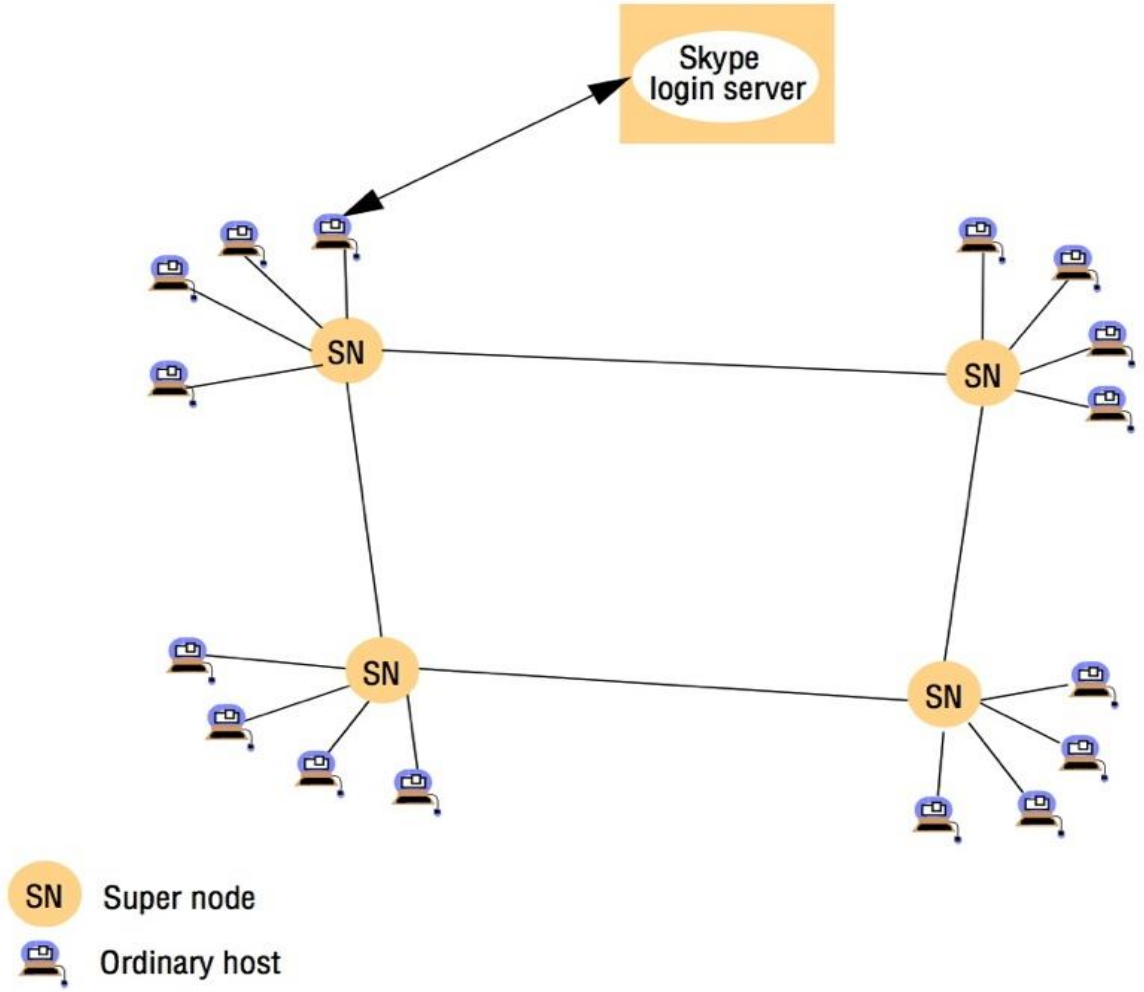
Gnutella Notes

- How do nodes find particular files
  - Initially used **flooding**, i.e. copy request to all nodes
    - Not scalable
- Added Query Routing Protocol
  - Nodes exchange summary index-like information about files on nodes
  - Queries only **forwarded** to possible matches
- (note – see also the discussion of distributed event services in the lecture on pub-sub/indirect communication)

Skype

- Peer-to-peer telephony
  - Developed by Kazaa, 2003
- **Hybrid** system
  - Centralised servers for log-in / payment
- Two-tier peer-to-peer network
  - **Supernodes**
    - Dynamically selected ordinary hosts with suitable network access (e.g. global IP address, good bandwidth, reliable)
  - Ordinary **hosts**
    - After initial login are directed to a supernode (maintain a cache of several supernodes to contact)

Skype overlay architecture



Q: what's the problem with lots of ordinary hosts as peers?

Skype Notes

- Proprietary protocols (highly obfuscated) but now well documented
  - E.g. Baset and Schulzrinne [2006] https://arxiv.org/abs/cs/0412017
- The global **index** of users is distributed across the **supernodes**
  - Contact and call requests are handled by the supernodes, not the centralised server(s)
- Voice data is sent directly between hosts where possible
  - May have to be relayed by supernodes if both hosts are behind NAT
  - Can use UDP or TCP as voice transport
    - Uses TCP for signalling / control protocol

Distributed hash tables (DHT)

- The network of peers maintains a set of **values**
  - E.g. files
- Each value is associated with a fixed-size **key**
  - (= the hash in a regular hash table)
  - E.g. a **secure hash** of an arbitrary key such as a file name
  - i.e. it is essential a hash table, but distributed ☺
- Peers can
  - **Add** a key/value to DHT
  - **Remove** a key/value from the DHT
  - **Get** a value from the DHT given its key
- E.g. Pastry https://en.wikipedia.org/wiki/Pastry_(DHT) …
  - See also Lecture on Distributed Algorithms

Peer-to-peer **middleware**

- Provides a flexible peer-to-peer facility (and/or overlay network) on which applications can be **built**
  - **Add** (and remove) resources (e.g. files)
  - **Route** requests to resources (e.g. read/retrieve)
  - **Join/leave** overlay network
    - Including coping with nodes that fail
- E.g.
  - **Distributed hash tables**, e.g. Pastry (see previous slide)
    - Simplest example
  - **Distributed object location and routing**, e.g. Tapestry
    - Add more flexibility, typically through more levels of indirection

Overlay Network Summary

An **overlay network** is a **virtual network** realised on top of an existing network such as the Internet, often with its own addresses, routing and characteristics(s), e.g.

- Tailored to a **specific application** or service
  - e.g. P2P
- Tailored for **challenging network environments**
  - e.g. wireless ad hoc
- Offering additional **network features**
  - e.g. security – VPNs, TOR

Peer-to-Peer Summary

- Peer-to-peer aim to be completely **decentralised**
  - Each user (node) contributes resources
  - All nodes have the same functional capabilities and responsibilities
  - Correct operation does not depend on any centrally administered system
- Peer-to-peer systems typically realised using an **application-specific overlay network**
  - See following examples…

Peer-to-Peer Examples Summary

- E.g. file sharing (Napster, Gnutella), teleconferencing (Skype)
- Note: often NOT pure peer-to-peer systems
  - May have regular nodes and ”super-nodes” (e.g. Gnuttella, Skype, to cope with NAT), and/or
  - May have centralised components as well as peers (e.g. Napstar, to bootstrap, Skype, for authentication & billing)
- Peer-to-peer **middleware** can provide a flexible peer-to-peer facility (or overlay network) on which applications can be built
  - E.g. **Distributed Hash Tables** (DHT, e.g. Pastry)
    - I.e. a hash table with values distributed across all participating nodes

Next steps

- Lecture 16: Broadcast, Multicast & discovery
- Lab 8: Java Multicast
- (Review & revise, past paper questions, labs, … :)